readcon-core: a reader and writer for the CON checkpoint format

Rohit Goswami[a,1]

[a] *TurtleTech ehf., Reykjavík, Iceland*

Abstract

Rare events in solids and molecules are too slow for a direct trajectory, so a rate calculation searches for saddle points and for the path that joins two minima. The search calls a force evaluator many times, passes geometries between codes, and restarts from a checkpoint. That checkpoint has to carry the cell, a constraint on each Cartesian axis, an atom identity that survives grouping by element, and velocities in a declared time unit. Here we present readcon-core, a Rust reader and writer for the CON checkpoint, with a C interface and bindings for C++, Python, Julia and Fortran. On a band of ten thousand frames the library is about 3 times faster than the ASE reader of the same text. The checks are a conformance corpus, round trips of producer files, and the public HCN to HNC path of a nudged elastic band. The program is released under the MIT licence.

Keywords: CON, checkpoint, nudged elastic band, saddle search, rare events

Program Summary

Program Title: readcon-core
CPC Library link to program files: (to be added by the Technical Editor)
Developer's repository link: `https://github.com/lode-org/readcon-core`
Licensing provisions: MIT
Programming language: Rust for the core, with a C header, a C++ header,

Email address: `rgoswami@ieee.org` (Rohit Goswami)
[1]ORCID: 0000-0002-2393-8056.

and bindings for Python, Julia and Fortran 2008
Supplementary material: the format specification, the conformance corpus, and the benchmark scripts and timing tables
Nature of problem: Harmonic transition state theory estimates a rare-event rate from the barrier and the curvatures at a minimum and at a first-order saddle [1, 2]. Finding those points, and the path that connects two minima, is the work of the nudged elastic band, its climbing-image form, the dimer method, and adaptive kinetic Monte Carlo [3, 4, 5, 6]. A campaign of that kind calls a force code thousands of times, writes a geometry after many of those calls, and starts again from the last good frame. The frame has to remember which Cartesian axes are held fixed, because a slab or a surface is rarely frozen in every direction. It has to remember which atom is which after the file groups atoms by chemical type, because images of a path are compared atom by atom. When the restart continues a trajectory, the velocities have to be in a known time unit, because the kinetic energy scales with the square of that unit. CON is the text checkpoint used for this state. *Solution method:* readcon-core is one parser and one writer for CON text. A frame is a short header, coordinates with a constraint flag and an atom id, and optional sections for velocities, forces and other per-atom fields. Line 2 of a current file is a single JSON object that records the major version, the minor version, the unit system and the section names. From the minor version that makes the time label authoritative, a velocities section is written only with an explicit time unit, and the natural unit shared with existing producers is `(amu*angstrom^2/eV)^0.5`. Decimal numbers are read with correct rounding and, unless the caller asks for a fixed number of digits, written so that they parse back to the same binary64 value. A C header is the narrow waist: C++, Python, Julia and Fortran call that waist, so each language sees one parse. Parallel ingest splits the file on frame boundaries that depend only on the text, and the frames come back in file order at every thread count. gzip and, when the optional feature is built, zstd are compression envelopes around the same text. *Additional comments including restrictions and unusual features:* The C interface reports a failed build or a failed read through a status code and does not keep an error string for a later call. The Python extension is not a free-threaded build. zstd input is accepted only when the library is built with that feature. A pool of threads is used automatically only for files above 48 KiB, and a file that holds a single frame is parsed as one task even if that frame is large. The same fixtures are exercised from the Rust core and from the C, Python, Julia and Fortran bindings. *References:*

[1] G. H. Vineyard, J. Phys. Chem. Solids 3 (1957) 121, doi:10.1016/0022-3697(57)90059-8.
[2] P. Hänggi, P. Talkner, M. Borkovec, Rev. Mod. Phys. 62 (1990) 251, doi:10.1103/RevModPhys.62.251.
[3] H. Jónsson, G. Mills, K. W. Jacobsen, in Classical and Quantum Dynamics in Condensed Phase Simulations, World Scientific, 1998, p. 385, doi:10.1142/9789812839664_0016.
[4] G. Henkelman, B. P. Uberuaga, H. Jónsson, J. Chem. Phys. 113 (2000) 9901, doi:10.1063/1.1329672.
[5] G. Henkelman, H. Jónsson, J. Chem. Phys. 111 (1999) 7010, doi:10.1063/1.480097.
[6] G. Henkelman, H. Jónsson, J. Chem. Phys. 115 (2001) 9657, doi:10.1063/1.1415500.

1. Introduction

Thermally activated events set the rates of diffusion, reactions and structural change, and they are rare on the clock of a direct molecular dynamics run. Harmonic transition state theory estimates such a rate from the barrier height and from the curvatures at the minimum and at a first-order saddle [1, 2]. The practical task is to find those stationary points and the path that joins two known minima. The searches that do that work fall into a double-ended family and a single-ended family. A double-ended search, the nudged elastic band and its climbing-image form, discretizes the path into a chain of images and relaxes that chain until the highest image sits near the saddle [3, 4, 7]. A single-ended search, the dimer method, rotates a pair of replicas until the lowest curvature points uphill and then walks toward the saddle from one basin [5]. Adaptive kinetic Monte Carlo accepts an escape, advances a physical clock by the waiting time of that escape, and repeats the search from the new minimum [6]. Energy-weighted springs, together with eigenvector following, keep a molecular band from cutting the corner of a sharply curved path [8]. An improved estimate of the tangent keeps the elastic band from forming kinks at places where the path bends sharply [9]. The image-dependent pair potential starts the chain of images closer to the minimum energy path the band will converge to [10]. The string method was introduced for the sampling of rare events as a curve that is evolved between two known states [11]. A later form of that method finds a minimum energy path by evolving the curve rather than by relaxing a chain of springs [12].

A comparison of those single-ended searches showed that a saddle can still be found when the product state is unknown [13]. A later dimer method converges faster than the original rotation because it uses a better estimate of the curvature [14].

These campaigns spend most of their effort on force calls, and they spend the rest on moving the current geometry from one program to another and on starting again after a stop. A reproducible band optimization calls a potential for every image at every iteration, and the checkpoint is what lets that run resume [15]. A tutorial review of Gaussian-process searches shows how a surrogate can cut the number of force evaluations on a stationary-point problem [16]. The same idea has been implemented for molecular saddle searches, where the surrogate still has to land on a force evaluator [17]. Adaptive pruning then drops training points that no longer change the next step, which keeps a long search from storing every geometry [18]. An earlier Gaussian-process band uses inverse interatomic distances so that a nudged elastic band needs fewer calls to the potential [19]. A Gaussian-process minimum-mode search uses the same kind of covariance to rotate a dimer toward a saddle from one basin [20]. Eigenmode alignment of the climbing image keeps the highest bead on the softest mode of the Hessian while the band relaxes [21]. The same files are read again when a path is projected onto two dimensions and compared with the band that produced it [22]. A long kinetic Monte Carlo run writes a checkpoint whenever it accepts a move, and that file has to be readable after the job has stopped [6]. Between those reads the file may sit on disk for hours while a queue waits, or it may be copied onto another machine that has a different build of the force code. The older eOn paper is the reference for the long-timescale code in which this checkpoint became the working file, and a restart is only as faithful as the frame that code wrote [23].

The checkpoint has to carry a per-axis mask, an atom id, and a time unit for any velocities. The mask has one bit for x, one for y and one for z, so a buried atom carries 7, a free surface atom carries 0, and an adsorbate whose height is held carries 4. The id has to survive grouping by chemical type, so that image i of a band still names the same atom as image $i + 1$. A velocity is stored with its time unit, because the kinetic energy scales with the square of that unit. The natural unit used by eOn and by ASE is $(\mathrm{amu} \cdot \mathrm{angstrom}^2/\mathrm{eV})^{0.5}$, which equals $10.1805057\,\mathrm{fs}$, and $1\,\mathrm{fs} = 10^{-15}\,\mathrm{s}$.

CON is the text form of that checkpoint, written so that a failed search can be inspected without a decoder. It grew up inside eOn, which writes

one frame per minimum, saddle or band image and reads it back on the next cycle [23]. The Atomic Simulation Environment reads and writes the same text, which is how a geometry moves from the saddle search into a Python workflow [24]. A band file is therefore a sequence of frames that several programs already agree to exchange.

Neighbouring formats store a trajectory or a structure, and the fields that differ from this checkpoint are the integer mask, the id that survives grouping inside the frame, and the versioned time label. H5MD stores positions and optional velocities in HDF5, defines an id element for each particle, and can attach a time unit as a dataset attribute [25, 26]. That id and that attribute cover identity and the clock for a molecular trajectory, while the integer mask and the versioned time label stay with this checkpoint. NetCDF is an array container, so any checkpoint convention built on it has to be agreed by the programs that share the file [27]. The TNG format stores coordinates, velocities, forces and a box, and it takes atom identity from the molecular topology [28]. ASE's extended XYZ writer stores a per-axis move_mask, one boolean for each Cartesian axis, which records a constraint in a different encoding from the integer mask in this file [24]. A LAMMPS dump can carry an atom id and, when the dump style asks for them, velocities and forces, while the unit style of the run stays outside the frame [29]. The LAMMPS data file holds the starting geometry and the dump holds the trajectory, so neither file is the band of images this checkpoint exchanges [29]. Zarr is a chunked array store, and its community standard defines how those chunks are laid out on disk [30]. CHARMM writes a DCD trajectory of coordinates and of the box, which is the record of a finished dynamics run [31]. GROMACS writes XTC and TRR trajectories, so a reader of those files gets positions and sometimes velocities from a molecular run [32]. MDTraj reads DCD and XTC when the task is analysis of a finished trajectory [33]. A protein structure is often stored as a PDB file, and atom identity there comes from the residue and the atom name [34]. mmCIF is the macromolecular archive form of that structure, with the same residue and atom-name identity [35]. A small-molecule crystal structure travels as a CIF, which archives the published cell and the published coordinates [36]. A plane-wave calculation reads a POSCAR geometry, the cell and the coordinates that VASP will relax [37]. pymatgen converts among the crystal files used in solid-state work, including POSCAR and CIF, and the result is still one geometry [38]. Atomsk converts among the files used for defects and for classical potentials, which is how a geometry changes dialect before a run [39]. Open Babel converts among the chemical

formats used for molecules, and those formats name bonds and residues [40]. OVITO displays atomistic snapshots and can read several of these files when the task is to look at a geometry [41]. i-PI drives a calculation by passing coordinates to a client and reading forces back, one geometry at a time [42]. That coordinate exchange began as a path-integral client, which sent positions out and received forces back [43]. A liquid or a mixture is often packed into a cell before the first minimization, and that packed geometry is what the checkpoint then stores [44]. Machine-learning potentials exchange atomic features as tensors, and a tensor of features is a different object from the geometry a search writes [45]. Table 1 puts the checkpoint next to those files, after the layout of one frame has been described.

Here we present readcon-core, a reader and writer for the CON checkpoint used by saddle searches. The library is a Rust parser and writer, exposed through one C header so that C++, Python, Julia and Fortran share one reading of the file. The parts a restart depends on are the versioned time label, the round-trip default for floating-point text, and the preservation of unknown metadata keys. The parallel read finds frame boundaries from the text alone, so the frames come back in file order at every worker count, as Figure 2 shows. The same program also opens the older files that eOn and ASE have been writing, including velocities stored in the natural unit. Section 2 describes the file, Section 3 describes the parser, and Section 4 reports the corpus, the round trip, the slab and the HCN path. Section 5 reports one-thread Python timings against one-thread peers, and it reports thread scaling from a separate run. Section 6 states what the release covers, and Section 7 collects the result of those checks.

2. The CON format

A CON file is a sequence of frames in plain text, so that a failed search can be inspected without a decoder, and Figure 1 shows how one frame is laid out. The first nine lines are the header: a generator note, a metadata line, three cell lengths, three cell angles, and two further text lines. The last three of those nine lines are the number of chemical types, the count of atoms of each type, and the mass of each type. Coordinates follow, grouped by chemical symbol, and each group is a symbol, a line that names the component, and one row per atom. A coordinate row holds x, y, z, the mask and the id, five tokens on one line, and a missing token or an extra token makes the row illegal. Optional sections repeat that grouping after a blank line, velocities

first when a velocity block is present. Lengths in the files discussed here are in angstrom, $1\,\text{Å} = 10^{-10}\,\text{m}$, and energies are in electronvolts.

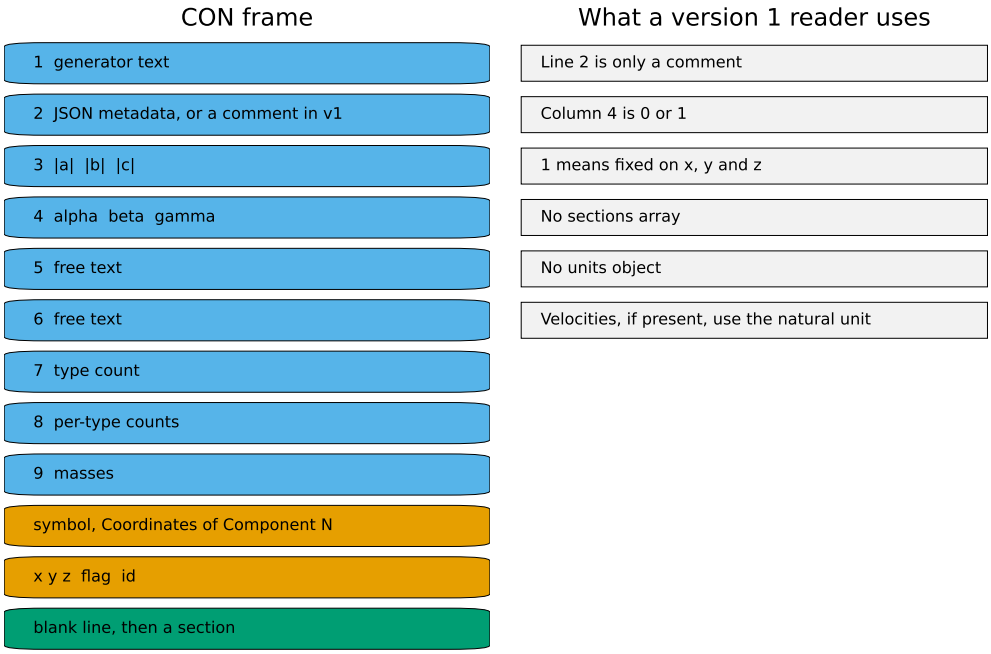


Figure 1: Left: the lines of one CON frame, from the generator note through an optional section. Right: the subset a version 1 reader uses. A version 1 file has a comment on line 2, treats column 4 as 0 or 1, and has no units object. Blue marks the header, orange the coordinate group, and green the section that follows a blank line.

The cell is three lengths and three angles in degrees, which is enough to rebuild the box once the angle convention is fixed. The angle on line 4 between b and c is α, the angle between a and c is β, and the angle between a and b is γ. The convention puts a along x and b in the xy plane, which is how both eOn and ASE turn those six numbers back into vectors [23, 24]. When the metadata also carries lattice vectors, those vectors are three rows of the cell matrix and they take priority on read, and the writer keeps lines 3 and 4 consistent with them. A missing periodicity flag means the cell is periodic in all three directions, which is the usual case for a bulk crystal and for a slab that is periodic in the plane. Column 4 is the mask from the introduction, stored as an integer token, and column 5 is the id. Version 1 stores 0 or 1 in the mask column, and from version 2 the column is the three-bit integer.

The reader rules for a flag outside that range, for a missing id, and for a repeated id are in Appendix B. The id is read as an integer up to $2^{64} - 1$, so a large identity is not rounded through a floating-point conversion. When the ids inside a frame are unique, sorting them recovers the order the atoms had before the file grouped them by chemical type. That order is what a distance or a projection expects when it compares one image of a band with the next.

Line 2 of a current file is one line of JSON, without a byte-order mark, and it records the major version, the minor version, the units and the section names. A reader accepts an unknown minor of a major it knows, and it refuses an unknown major, so a later clarification can travel in a file that an older program of the same major still opens. Unknown keys on that line are kept when the frame is written again, so a campaign can attach an energy, a bead index or a potential name without waiting for every reader to learn the key. The time unit on the metadata line is the label already stated in the introduction, and the writer copies that spelling. From the minor version that records time, the label is authoritative, and a writer that emits velocities must emit the time unit with them. On older files the velocities, when a velocity block is present, are in the natural unit, because that is the unit those producers wrote. A writer that is given a time unit it cannot parse fails at the moment the frame is built, and the library keeps the natural-unit spelling so that a later textual comparison still matches. Length and energy are required once the file is major version 3, and mass is recorded on the same object.

Sections are named blocks that follow the coordinates, and each block repeats the grouping by chemical symbol. A velocities section, a forces section, and further blocks for charges, spins, magnetic moments, displacements and spreads follow the same pattern as the coordinates: a symbol, a heading, and one row per atom. Exactly one blank line stands before each section, and no blank line stands between the component blocks inside a section, which is how a new name is recognized as a new section. The metadata lists the names that are present, so a reader knows what to expect before the first row of a section, and a listed name that never arrives is an incomplete frame. A count of zero chemical types is a legal empty frame, with no coordinate block and an empty section list. A header that stops early, or a section with fewer rows than the header declared, is rejected, and the partial frame is not returned. The remaining reader rules, including unknown section names, non-finite numbers and a gzip archive with several members, are the rows of

Appendix B.

Readers convert every decimal field to binary64 with correct rounding, which is the numeric contract a second parse relies on [46, 47]. The default writer prints each finite value so that reading it back yields the same binary64 number, which is the shortest-round-trip problem solved by Steele and White [48]. Grisu prints a floating-point value using only integer arithmetic so that the decimal form is short and correctly rounded [49]. Ryu prints a floating-point value by a table-driven procedure so that the decimal form is short and correctly rounded [50]. The metadata line is JSON, and a scanner for that kind of text has been shown to run at a very high rate on such input [51]. A caller may ask for a fixed number of digits after the decimal point, and that mode is lossy: values that differed in the last bits can collide, and values that needed more digits no longer parse back to themselves. gzip and zstd are envelopes around the same bytes, and the bytes that come out of either envelope are parsed by the same reader as an uncompressed file.

3. Implementation

The core is a Rust library with a single parser, and file reads, string reads and the bindings all go through that parser, so a change in the grammar shows up once. The parser walks the header, checks the counts, reads each coordinate row as an integer flag and an integer id, and then reads the sections named in the metadata. Decimal fields in that parser go through a fast correctly rounded conversion, the same contract the writer relies on [47]. Keeping one parser is what makes the conformance story possible, because a second implementation in each language would be a second place for a mask or a time label to drift. The C header owns the frame on the far side of the call and returns a status code, and the caller frees what it was given. C++ is a header over that interface, and Julia and Fortran call the same symbols, with the Fortran side using the standard C binding. Python uses the same parse, so a file that is legal in one binding is legal in the others, and a file that is rejected is rejected for the same reason. The status code is the whole of the error report on the C side, and Section 6 states that boundary as part of the release. Every language maps one integer, and none of them invents a second message for the same failure.

A caller builds a frame from a cell, symbols, coordinates, the mask, the id, and any velocities or forces. On a current frame the builder writes the minor number when the caller did not set one, and it writes a units object when the

caller did not supply one. Velocities are written only with the time unit the caller set, which is the rule that keeps a restarted kinetic energy on the clock that was stored. If the frame has velocities and the minor version requires a time unit, the build fails unless that unit was set. When an older frame that already contains velocities is rewritten, the natural unit is recorded on the new metadata line, so the next reader sees the clock those components used. The default text of every floating-point field is the round-trip form, and fixed digits remain a separate request for a caller who wants a shorter file and accepts the loss.

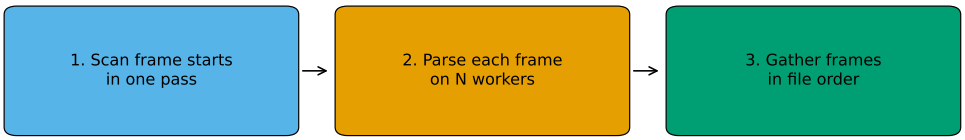


Figure 2: Read of a multi-frame file. A serial scan records where each frame starts. Each frame is then parsed on one of N workers. The frames are returned in file order.

Figure 2 is the parallel read of a band, which is already a list of frames rather than one array of atoms. A serial scan finds the byte offset where each frame starts, using the header counts, and that scan does not build the atoms. The pool then parses each recorded slice on its own worker, and the frames are assembled in the order the scan found them. The offsets depend only on the bytes of the file, so the split is the same however many workers are waiting. Below 48 KiB the read stays on the serial parser, because the cost of starting the pool is larger than the parse. A file with one frame is one task, however many atoms that frame contains, because the split is on frames and not on atoms. The Python call can pin the worker count: one worker forces the serial path, and two or more pin a pool of that size. The result is the same sequence of frames at every count, which Section 5 checks by hashing the stored fields, including the mask, the id and the velocities. The Python call releases the interpreter lock while the file is parsed, and it takes the lock back to build the objects the caller sees, so other Python work can run during a long parse.

The Python surface can also build a frame from an ASE atoms object, copying the geometry into the frame type the parser returns. That copy

takes positions, the cell, masses, a fully fixed atom where ASE stored that constraint, a per-axis mask where ASE stored one, and velocities when ASE has them. The numbers are copied as ASE holds them, so a caller who has velocities in a known unit sets that unit on the builder, and the components stay in the unit the caller already chose. The reverse direction writes that frame back out as the CON text the ASE eon reader expects. chemfiles is an optional ingress for formats other than CON, and a build that includes it can open an XYZ file into the same frame type [52]. The CON text remains the checkpoint that eOn and ASE exchange, and the other formats are a way in.

A campaign touches the same frame more than once, which is why these checks run at each touch. The builder records the cell, the mask and the id before the first force call, and the writer emits the text a queue will store. A later rank reads that text, compares the ids, and calls the potential on the free axes only. If that rank writes the frame again, unknown keys such as a bead index or a stored energy are still on the metadata line. The round-trip printer is what keeps the coordinates on that second write, because a short fixed-digit print would move atoms by a last-bit rounding on every restart.

4. Validation

The conformance corpus has 10 legal files and 8 files that must be rejected, covering the header, both meanings of the mask column, an empty frame, the units line, and files that stop early. The same files are read from 5 bindings, the Rust core and the C, Python, Julia and Fortran bindings, so a binding cannot quietly accept a file the core rejects. A legal file that one binding dropped, or an illegal file that one binding accepted, would show up as a different status on that binding. Table 1 places this checkpoint next to the ASE eon reader, extended XYZ, H5MD and a LAMMPS dump. What differs, once the extended XYZ move_mask and the H5MD id element are taken into account, is the integer mask in this file, the id that survives grouping inside the frame, and the versioned time label [24, 25]. The ASE eon reader follows the original eOn layout: a comment on the second line, a single fixed flag, and no velocity section.

A 218-atom copper-hydrogen frame from the eOn tests is read, written with the default printer, and read again, and every coordinate bit matches the value that was stored. The same coordinates match the ASE eon reading of that file, bit for bit, so the two programs agree on the positions of a

Table 1: What a restarted rare-event calculation needs, and what each file actually stores. The CON column is what readcon-core keeps.

Item	CON	ASE eon	ext. XYZ	H5MD	LAMMPS dump
mask	bits 0 to 7	one boolean	per-axis move_mask	not defined	not in the dump
id	kept in the row	not kept	optional column	id element	atom id
time unit	versioned label	not read	not required	dataset attribute	outside the frame
sections	named blocks	coordinates only	extra columns	particle groups	optional columns
version	major and minor	comment line	none	container version	none

producer file. The single-frame geometry used below and in Section 5 is a platinum (111) slab of 10001 atoms, built with ASE rather than by repeating one coordinate row. The cell is an orthogonal fcc111 slab of 50 by 20 by 10 layers, with 8 angstrom of vacuum, plus one hydrogen placed 1.5 angstrom above an on-top site. The bottom two layers are fixed on every axis, the other platinum atoms are free, and the adsorbate is fixed on z, which is the mask case a surface calculation actually writes. Table 2 shows one buried atom, one surface atom and the adsorbate from that slab, each with its own mask and its own id.

4.1. A time unit and a kinetic energy

The velocities on that slab are a Maxwell-Boltzmann sample at 300 K, drawn with ASE on a NumPy Generator and seed 20261011, and the kinetic energy is computed from those components. The energy is one half the sum of mass times speed squared, with the speeds in the natural unit, and on this frame it is 390.1 eV. Reading those same numbers as though they were angstrom per femtosecond multiplies the energy by 103.64 and gives 40429.5 eV. The two energies differ by 103.64, the square of that factor, because kinetic energy depends on the square of each velocity component. From the minor version that stores the time unit, the label selects the clock, and on older files the reader uses the natural unit those producers wrote.

4.2. HCN to HNC

The public HCN and HNC endpoints of an eOn nudged elastic band are the geometries used to illustrate a reproducible path optimization [15]. Those endpoints are rewritten here as current CON, with the natural time unit declared and a velocities section added. A third frame keeps the HCN coordinates, fixes hydrogen on x only, and stores the ids 2, 0 and 1 in file order. The file still groups the atoms as carbon, then nitrogen, then hydrogen, so the third row is hydrogen even though its id is 1 and the carbon in the first row carries id 2. A caller who sorts on the id column recovers the order the path tool expects, and a caller who zips the rows as they appear pairs the wrong atoms. Table 2 lists each atom of the HCN and HNC endpoints, of the probe, and of the generated slab. readcon-core returns the mask, the id and the velocities that were written, including the permutation 2, 0, 1 on the probe. On the coordinate form of the probe, which has no velocity section, the ASE eon reader reports the x-only hydrogen as fixed on every axis, which is the single flag of the original layout.

Table 2: Per-atom mask, id and velocity. A mask is three bits in order xyz. The probe stores ids 2, 0 and 1 and fixes hydrogen on x only. The slab rows are one buried atom, one surface atom and the adsorbate.

Atom	Reader	Mask	Id	Velocity
HCN C	readcon-core	000	0	present
HCN N	readcon-core	000	1	present
HCN H	readcon-core	000	2	present
HNC C	readcon-core	000	0	present
HNC N	readcon-core	000	1	present
HNC H	readcon-core	000	2	present
probe C	readcon-core	000	2	present
probe N	readcon-core	000	0	present
probe H	readcon-core	100	1	present
probe C	ASE eon	free	absent	absent
probe N	ASE eon	free	absent	absent
probe H	ASE eon	all-fixed	absent	absent
slab bottom Pt	readcon-core	111	0	present
slab surface Pt	readcon-core	000	9000	present
slab adsorbate H	readcon-core	001	10000	present

5. Performance

We compare the Python binding of readcon-core, at one worker, with four Python readers, each also at one thread, on one Intel Xeon Processor with 8 hardware threads. The peers are the ASE eon reader on the same CON text, a chemfiles read of an XYZ file with the same coordinates, an MDAnalysis read of an H5MD file with those coordinates, and an h5py read of that file's position array [24, 52, 53, 54]. OpenMP, BLAS and the related math libraries are held at one thread for the whole run, and that limit is set before any of those libraries is imported. The payloads are 1, 100 and 10 000 frames of a 218-atom copper-hydrogen geometry, and the platinum slab of Section 4. Each cell is 30 paired repeats, interleaved so that a slow drift hits both sides of the pair, and the ratio is the readcon-core time divided by the peer time. We report the median and a bias-corrected 95 percent interval, and we apply a Holm adjustment across the cells that produced a ratio. The full protocol, including the bootstrap seed and the Holm adjustment, is in Appendix A, and Figure 3 shows the intervals.

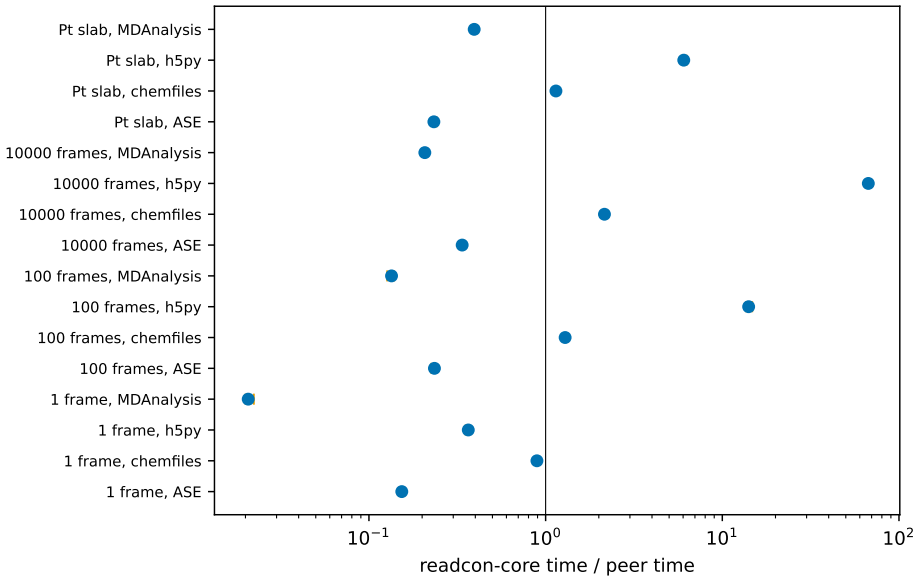


Figure 3: Median ratio of one-worker readcon-core time to one-thread peer time. A ratio below 1 means readcon-core finished sooner. The bars are bias-corrected 95 percent intervals of the median. The vertical line is a ratio of 1.

Against the ASE eon reader the speedup, the reciprocal of the ratio, runs

from 3 to 6.5 on the files both programs finish. Against chemfiles on the XYZ file with the same coordinates the speedup runs from 0.46 to 1.1, and a value below 1 means chemfiles finished sooner. Against MDAnalysis on the H5MD file the speedup runs from 2.5 to 48. On the band of 10 000 frames, readcon-core is 3 times faster than ASE and 4.8 times faster than MDAnalysis. On that same band the readcon-core time is 2.2 times the chemfiles time. h5py is faster on that band: the readcon-core time is 67 times the h5py time, which is the direction expected when the peer reads one dense array and readcon-core parses text. On a single small frame the comparison with h5py goes the other way, and readcon-core is 2.7 times faster, because the cost of opening the container dominates a tiny array. On the platinum slab, which is one frame and therefore one task, readcon-core is 4.3 times faster than ASE, and the h5py ratio on that same frame is 6.1.

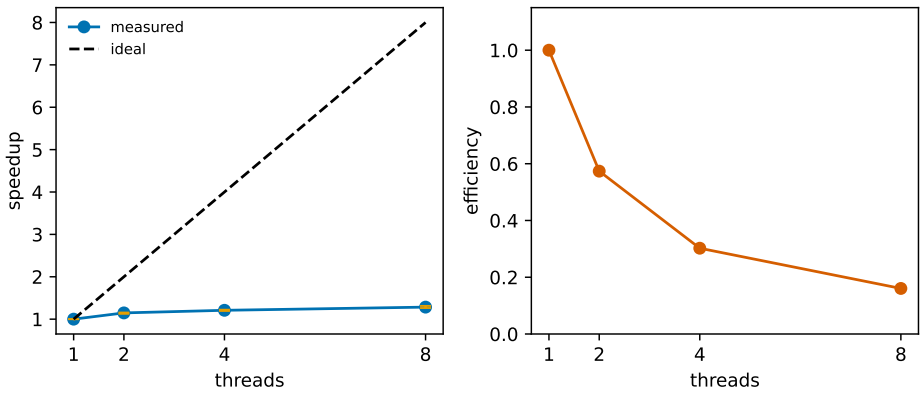


Figure 4: Thread scaling on the ten thousand frame file, from an interleaved run. Each multi-worker time is paired with a fresh one-worker baseline. Left: speedup, with the ideal line dashed and the interval of the median drawn. Right: parallel efficiency. The field hash is the same at 1, 2, 3, 4 and 8 threads.

Figure 4 is a separate run, and the speedup in that figure uses only the pairs recorded by the scaling loop. For each of 2, 4 and 8 workers the loop times one worker, then that worker count, thirty times, and the speedup is the ratio of those paired times. Two workers give a speedup of 1.15 (interval 1.13 to 1.16), four give 1.21 (1.19 to 1.22), and eight give 1.28 (1.26 to 1.31). Each of those speedups lies below the ideal line. The boundary scan that finds the frame offsets is serial, and each frame is a small parse, so the extra workers share a shrinking amount of independent work as the count rises.

The field hash is the same at 1, 2, 3, 4 and 8 threads.The frames that come back are the frames that were stored, at every worker count the hash covers.

6. Scope

The release reads and writes CON text, through one parser and one C header, with bindings for C++, Python, Julia and Fortran. The C call reports success or failure as a status code, and the caller maps that integer in whichever language sits above the header. The Python extension follows ordinary interpreter locking, and the parse itself runs without holding that lock, so other Python work can proceed during a long read. zstd envelopes are part of a build that includes that feature, and the same text comes out of a gzip envelope on every build. Parallel work is the set of frames, so one large frame stays one task, which is the slab timing above. Fixed-digit printing is available for a caller who wants a shorter file, and a geometry that must come back unchanged stays on the default writer.

7. Conclusions

readcon-core is a reader and writer for the CON checkpoint used by eOn and ASE, with one parser behind a C header and bindings for C++, Python, Julia and Fortran. The versioned time label keeps a restarted kinetic energy equal to the stored one, which on the generated platinum slab is the difference between 390.1 eV and 40429.5 eV. The HCN to HNC endpoints and the slab atoms in Table 2 come back with the mask and the id that were written, including the per-atom bits on the buried atom, the surface atom and the adsorbate. On the 10 000 frame band, at one worker, the Python binding is 3 times faster than the ASE eon reader, while h5py remains faster on a dense position array. Thread scaling is reported from its own interleaved run, and the hashed fields match across the worker counts that run covers. readcon-db is a downstream store that indexes this text for a search over many checkpoints.

Data availability

The software archive is `https://doi.org/10.5281/zenodo.18753567`, and the program will be deposited in the CPC Program Library [55]. A numbered release of the library is archived beside the concept record, so one

version can be cited as well as the series [56]. The archive is the software object that the FAIR principles for research software ask an author to identify and to describe [57]. A citation to research software is owed to the archived record, which is why the concept DOI appears in this paper [58]. The benchmark scripts and the per-repeat timing tables are part of the submission archive. The HCN and HNC endpoints are the public files of the nudged elastic band study [15]. The public endpoints are written by the orchestrator that drives the band, and the file it stores is the checkpoint this library reads [59].

CRediT authorship contribution statement

Rohit Goswami: Conceptualization, Methodology, Software, Validation, Visualization, Writing - original draft, Writing - review & editing.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work received no funding.

Acknowledgements

The people who shaped this checkpoint, and the author's family, will be named here once that list is confirmed.

Appendix A. Benchmark protocol and ratios

The host label in the timing tables is bench-vm, and the processor, the hardware-thread count, the compiler, the library revision, the script revision and the UTC date are columns of the per-repeat table. The comparison calls the Python binding with one worker, and ASE, chemfiles, MDAnalysis and h5py run in the same process with OpenMP and BLAS limited to one thread. The 1-frame, 100-frame and 10 000-frame files repeat one 218-atom copper-hydrogen frame, and the slab is a separate single frame. The slab is

built with ASE 3.29 as an orthogonal fcc111 platinum cell of size 50 by 20 by 10, vacuum 8 angstrom, plus one hydrogen adsorbate at height 1.5 angstrom on an on-top site. The bottom two layers are fixed on every axis, the other platinum atoms are free, and the adsorbate is fixed on z. Velocities for the kinetic-energy check are a Maxwell-Boltzmann sample at 300 K on that slab, drawn with a NumPy Generator and seed 20261011, and that velocity section is left out of the timing file so the ASE eon reader can finish the frame. XYZ and H5MD peers receive the coordinates of the same frames, written beside the CON text before the timed loop starts. For each payload and each peer the loop is one-worker readcon-core, then the peer, thirty times, after one untimed call of each side. A repeat that exceeds 240 seconds is recorded as a timeout, and a peer that raises is recorded as a failure, and neither of those outcomes is turned into a time. The ratio uses only pairs in which both sides returned a time, and the interval is a bias-corrected and accelerated bootstrap of the median, with 9999 draws and seed 20261011. A two-sided bootstrap probability for the median ratio equalling 1 is adjusted by the Holm step-down across those cells. Thread scaling is a second loop on the 10 000-frame file, and for 2, 4 and 8 workers each repeat times one worker and then that worker count. The field hash of the stored coordinates, masks and ids is taken at 1, 2, 3, 4 and 8 workers.

Appendix B. Reader rules

The rules below are the edge cases of the reader. The mask, the id and the time unit themselves are stated in the introduction.

References

[1] G. H. Vineyard, Frequency factors and isotope effects in solid state rate processes, Journal of Physics and Chemistry of Solids 3 (1) (1957) 121–127. doi:10.1016/0022-3697(57)90059-8.

[2] P. Hänggi, P. Talkner, M. Borkovec, Reaction-rate theory: fifty years after Kramers, Reviews of Modern Physics 62 (2) (1990) 251–341. doi:10.1103/RevModPhys.62.251.

[3] H. Jónsson, G. Mills, K. W. Jacobsen, Nudged elastic band method for finding minimum energy paths of transitions, in: B. J. Berne, G. Ciccotti, D. F. Coker (Eds.), Classical and Quantum Dynamics

Table A.3: Paired time ratios. The ratio is the readcon-core time divided by the peer time, so a value below 1 means readcon-core finished sooner. The interval is a bias-corrected and accelerated 95 percent interval of the median. Failures stay in the table and are not converted into times.

Payload	Peer	OK	Fail	Ratio	Interval
1 frame	ASE eon	30	0	0.153	[0.15, 0.155]
1 frame	chemfiles XYZ	30	0	0.893	[0.888, 0.902]
1 frame	h5py	30	0	0.365	[0.353, 0.37]
1 frame	MDAnalysis H5MD	30	0	0.0207	[0.02, 0.0223]
100 frames	ASE eon	30	0	0.235	[0.226, 0.238]
100 frames	chemfiles XYZ	30	0	1.29	[1.26, 1.32]
100 frames	h5py	30	0	14.1	[13.6, 14.8]
100 frames	MDAnalysis H5MD	30	0	0.134	[0.127, 0.139]
10000 frames	ASE eon	30	0	0.337	[0.335, 0.345]
10000 frames	chemfiles XYZ	30	0	2.15	[2.14, 2.17]
10000 frames	h5py	30	0	67.1	[66.5, 67.3]
10000 frames	MDAnalysis H5MD	30	0	0.207	[0.204, 0.213]
Pt slab	ASE eon	30	0	0.233	[0.231, 0.241]
Pt slab	chemfiles XYZ	30	0	1.14	[1.13, 1.16]
Pt slab	h5py	30	0	6.05	[5.91, 6.27]
Pt slab	MDAnalysis H5MD	30	0	0.394	[0.387, 0.404]

in Condensed Phase Simulations, World Scientific, 1998, pp. 385–404. doi:10.1142/9789812839664_0016.

[4] G. Henkelman, B. P. Uberuaga, H. Jónsson, A climbing image nudged elastic band method for finding saddle points and minimum energy paths, The Journal of Chemical Physics 113 (22) (2000) 9901–9904. doi:10.1063/1.1329672.

[5] G. Henkelman, H. Jónsson, A dimer method for finding saddle points on high dimensional potential surfaces using only first derivatives, The Journal of Chemical Physics 111 (15) (1999) 7010–7022. doi:10.1063/1.480097.

[6] G. Henkelman, H. Jónsson, Long time scale kinetic Monte Carlo simulations without lattice approximation, The Journal of Chemical Physics 115 (21) (2001) 9657–9666. doi:10.1063/1.1415500.

[7] D. Sheppard, R. Terrell, G. Henkelman, Optimization methods for find-

Table B.4: Reader rules for flags, versions and incomplete input.

Input	Rule
Mask outside the integers 0 to 7	The row is rejected.
Version 1 mask equal to 1	x, y and z are all fixed.
Missing id column	The id is the zero-based row index.
Repeated id	The row order in the file is kept.
Unknown major version	The file is rejected.
Unknown minor of a known major	The file is accepted.
Unknown metadata key	The key is kept when the frame is written again.
Unknown section at this minor	The file is rejected.
Unknown section at a newer minor, with a declared width	The block may be skipped.
Non-finite number	The field is rejected.
Zero chemical types	The frame is legal and empty.
Truncated header or a short section	The partial frame is not returned.
gzip archive with several members	Every member is read, in order.

ing minimum energy paths, The Journal of Chemical Physics 128 (13) (2008). doi:10.1063/1.2841941.

[8] V. Asgeirsson, et al., Nudged elastic band method for molecular reactions using energy-weighted springs combined with eigenvector following (2021). doi:10.1021/acs.jctc.1c00462.

[9] G. Henkelman, H. Jonsson, Improved tangent estimate in the nudged elastic band method for finding minimum energy paths and saddle points (2000). doi:10.1063/1.1323224.

[10] S. Smidstrup, A. Pedersen, K. Stokbro, H. Jonsson, Improved initial guess for minimum energy path calculations (2014). doi:10.1063/1.4878664.

[11] W. E, W. Ren, E. Vanden-Eijnden, String method for the study of rare events (2002). doi:10.1103/PhysRevB.66.052301.

[12] W. E, W. Ren, E. Vanden-Eijnden, Simplified and improved string

method for computing the minimum energy paths in barrier-crossing events (2007). doi:10.1063/1.2720838.

[13] R. A. Olsen, G. J. Kroes, G. Henkelman, et al., Comparison of methods for finding saddle points without knowledge of the final states (2004). doi:10.1063/1.1809574.

[14] J. Kastner, P. Sherwood, Superlinearly converging dimer method for transition state search (2008). doi:10.1063/1.2815812.

[15] R. Goswami, Reproducible orchestration of best practices for reaction path optimization with the nudged elastic band, MethodsX 16 (2026) 103899. doi:10.1016/j.mex.2026.103899.

[16] R. Goswami, A tutorial review of bayesian optimization with gaussian processes to accelerate stationary point searches (2026). doi:10.1021/acsphyschemau.6c00038.

[17] R. Goswami, M. Masterov, S. Kamath, A. Peña-Torres, H. Jónsson, Efficient implementation of Gaussian process regression accelerated saddle point searches with application to molecular reactions, Journal of Chemical Theory and Computation 21 (16) (2025) 7935–7943. doi:10.1021/acs.jctc.5c00866.

[18] R. Goswami, H. Jónsson, Adaptive pruning for increased robustness and reduced computational overhead in Gaussian process accelerated saddle point searches, ChemPhysChem 27 (4) (2026). doi:10.1002/cphc.202500730.

[19] O.-P. Koistinen, V. Asgeirsson, A. Vehtari, H. Jonsson, Nudged elastic band calculations accelerated with gaussian process regression based on inverse interatomic distances (2019). doi:10.1021/acs.jctc.9b00692.

[20] O.-P. Koistinen, V. Asgeirsson, A. Vehtari, H. Jonsson, Minimum mode saddle point searches using gaussian process regression with inverse-distance covariance function (2019). doi:10.1021/acs.jctc.9b01038.

[21] R. Goswami, M. Gunde, H. Jónsson, Enhanced climbing image nudged elastic band method with Hessian eigenmode alignment, Frontiers in Chemistry 14 (2026). doi:10.3389/fchem.2026.1807063.

[22] R. Goswami, Two-dimensional RMSD projections for reaction path visualization and validation, MethodsX 16 (2026) 103851. doi:10.1016/j.mex.2026.103851.

[23] S. T. Chill, M. Welborn, R. Terrell, L. Zhang, J.-C. Berthet, A. Pedersen, H. Jónsson, G. Henkelman, EON: software for long time simulations of atomic scale systems, Modelling and Simulation in Materials Science and Engineering 22 (5) (2014) 055002. doi:10.1088/0965-0393/22/5/055002.

[24] A. H. Larsen, J. J. Mortensen, J. Blomqvist, I. E. Castelli, R. Christensen, M. Dułak, J. Friis, M. N. Groves, B. Hammer, C. Hargus, E. D. Hermes, P. C. Jennings, P. B. Jensen, J. Kermode, J. R. Kitchin, E. L. Kolsbjerg, J. Kubal, K. Kaasbjerg, S. Lysgaard, J. B. Maronsson, T. Maxson, T. Olsen, L. Pastewka, A. Peterson, C. Rostgaard, J. Schiøtz, O. Schütt, M. Strange, K. S. Thygesen, T. Vegge, L. Vilhelmsen, M. Walter, Z. Zeng, K. W. Jacobsen, The atomic simulation environment—a Python library for working with atoms, Journal of Physics: Condensed Matter 29 (27) (2017) 273002. doi:10.1088/1361-648X/aa680e.

[25] P. de Buyl, P. H. Colberg, F. Höfling, H5MD: a structured, efficient, and portable file format for molecular data, Computer Physics Communications 185 (6) (2014) 1546–1553. doi:10.1016/j.cpc.2014.01.018.

[26] M. Folk, G. Heber, Q. Koziol, et al., An overview of the hdf5 technology suite and its applications, 2011. doi:10.1145/1966895.1966900.

[27] R. Rew, G. Davis, NetCDF: an interface for scientific data access, IEEE Computer Graphics and Applications 10 (4) (1990) 76–82. doi:10.1109/38.56302.

[28] M. Lundborg, R. Apostolov, D. Spångberg, A. Gärdenäs, D. van der Spoel, E. Lindahl, An efficient and extensible format, library, and API for binary trajectory data from molecular simulations, Journal of Computational Chemistry 35 (3) (2014) 260–269. doi:10.1002/jcc.23495.

[29] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, S. J.

Plimpton, LAMMPS — a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, Computer Physics Communications 271 (2022) 108171. doi:10.1016/j.cpc.2021.108171.

[30] Open Geospatial Consortium, Zarr storage specification 2.0 community standard. doi:10.62973/21-050r1.
URL https://doi.org/10.62973/21-050r1

[31] B. R. Brooks, et al., Charmm: The biomolecular simulation program (2009). doi:10.1002/jcc.21287.

[32] M. J. Abraham, et al., Gromacs: High performance molecular simulations through multi-level parallelism from laptops to supercomputers (2015). doi:10.1016/j.softx.2015.06.001.

[33] R. T. McGibbon, et al., Mdtraj: A modern open library for the analysis of molecular dynamics trajectories (2015). doi:10.1016/j.bpj.2015.08.015.

[34] H. M. Berman, The protein data bank (2000). doi:10.1093/nar/28.1.235.

[35] J. D. Westbrook, P. E. Bourne, Star/mmcif: An ontology for macromolecular structure (2000). doi:10.1093/bioinformatics/16/2/159.

[36] S. R. Hall, F. H. Allen, I. D. Brown, The crystallographic information file (cif): a new standard archive file for crystallography (1991). doi:10.1107/S010876739101067X.

[37] G. Kresse, J. Furthmuller, Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set (1996). doi:10.1016/0927-0256(96)00008-0.

[38] S. P. Ong, et al., Python materials genomics (pymatgen): A robust, open-source python library for materials analysis (2013). doi:10.1016/j.commatsci.2012.10.028.

[39] P. Hirel, Atomsk: A tool for manipulating and converting atomic data files (2015). doi:10.1016/j.cpc.2015.07.012.

[40] N. M. O'Boyle, et al., Open babel: An open chemical toolbox (2011). doi:10.1186/1758-2946-3-33.

[41] A. Stukowski, Visualization and analysis of atomistic simulation data with ovito-the open visualization tool (2009). doi:10.1088/0965-0393/18/1/015012.

[42] V. Kapil, et al., i-pi 2.0: A universal force engine for advanced molecular simulations (2019). doi:10.1016/j.cpc.2018.09.020.

[43] M. Ceriotti, J. More, D. E. Manolopoulos, i-pi: A python interface for ab initio path integral molecular dynamics simulations (2014). doi:10.1016/j.cpc.2013.10.027.

[44] L. Martinez, R. Andrade, E. G. Birgin, et al., Packmol: A package for building initial configurations for molecular dynamics simulations (2009). doi:10.1002/jcc.21224.

[45] F. Bigi, et al., metatensor and metatomic: Foundational libraries for interoperable atomistic machine learning (2026). doi:10.1063/5.0304911.

[46] W. D. Clinger, How to read floating point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation, 1990, pp. 92–101. doi:10.1145/93542.93557.

[47] D. Lemire, Number parsing at a gigabyte per second, Software: Practice and Experience 51 (8) (2021) 1700–1727. doi:10.1002/spe.2984.

[48] G. L. Steele Jr., J. L. White, How to print floating-point numbers accurately, in: Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation, 1990, pp. 112–126. doi:10.1145/93542.93559.

[49] F. Loitsch, Printing floating-point numbers quickly and accurately with integers, 2010. doi:10.1145/1806596.1806623.

[50] U. Adams, Ryu: fast float-to-string conversion, 2018. doi:10.1145/3192366.3192369.

[51] G. Langdale, D. Lemire, Parsing gigabytes of json per second (2019). doi:10.1007/s00778-019-00578-5.

[52] G. Fraux, et al., chemfiles (2023). doi:10.5281/zenodo.1202207.
URL https://doi.org/10.5281/zenodo.1202207

[53] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, O. Beckstein, MDAnalysis: a toolkit for the analysis of molecular dynamics simulations, Journal of Computational Chemistry 32 (10) (2011) 2319–2327. doi:10.1002/jcc.21787.

[54] R. J. Gowers, M. Linke, J. Barnoud, T. J. E. Reddy, M. N. Melo, S. L. Seyler, J. Domański, D. L. Dotson, S. Buchoux, I. M. Kenney, O. Beckstein, MDAnalysis: a Python package for the rapid analysis of molecular dynamics simulations, in: Proceedings of the 15th Python in Science Conference, 2016, pp. 98–105. doi:10.25080/Majora-629e541a-00e.

[55] R. Goswami, readcon-core: a CON and convel reader and writer with a C interface (2026). doi:10.5281/zenodo.18753567.
URL https://doi.org/10.5281/zenodo.18753567

[56] R. Goswami, readcon-core: a con/convel reader/writer with ffi bindings (2026). doi:10.5281/zenodo.23264985.
URL https://doi.org/10.5281/zenodo.23264985

[57] M. Barker, et al., Introducing the fair principles for research software (2022). doi:10.1038/s41597-022-01710-x.

[58] A. M. Smith, D. S. Katz, K. E. Niemeyer, Software citation principles (2016). doi:10.7717/peerj-cs.86.

[59] R. Goswami, eon orchestrator (2026).
URL https://github.com/HaoZeke/eon_orchestrator